

From Distribution-Free CDF Bands to Polyhedral Confidence Regions for Location–Scale Families

A documented AI-generated research artifact

Roman Guchenko

Human initiator, curator, and verifier

Substantive scientific development and initial manuscript generation

ChatGPT (OpenAI, GPT-5.6 Sol)

Research provenance. This work arose from an extended research dialogue between the named human participant and ChatGPT. The human participant posed the initial research question, selected among directions proposed during the investigation, executed computational experiments, returned their results to the model, later personally reviewed and verified the mathematical arguments and R implementation, curated the resulting research record, and prepared the work for public dissemination.

ChatGPT generated most of the subsequent substantive scientific development, including literature exploration, formulation and generalization of the statistical framework, mathematical derivations, algorithm design, R code drafts, simulation design, interpretation of results, and the initial manuscript text. The named human therefore does not claim conventional authorship of those AI-generated contributions. His role in this publication is stated explicitly as human initiator, curator, and verifier.

The research-development transcript, source code, computational notebook, verification materials, and manuscript source are preserved with this paper so that the provenance of the work can be examined directly.

Research developed August 2026
Public version: 20 September 2026
Research record: `[repositoryURL]`

Abstract

This paper studies confidence regions for the location and scale parameters of a continuous location–scale family obtained by inverting simultaneous, distribution-free confidence bands for the cumulative distribution function. If the band implies simultaneous order-statistic constraints

$$\ell_i \leq F(X_{(i)}) \leq u_i, \quad i = 1, \dots, n,$$

then for a location–scale family

$$F_{\mu,\sigma}(x) = G\left(\frac{x - \mu}{\sigma}\right), \quad \sigma > 0,$$

the induced confidence region is the intersection of the affine constraints

$$X_{(i)} - \sigma G^{-1}(u_i) \leq \mu \leq X_{(i)} - \sigma G^{-1}(\ell_i).$$

Consequently, the region is convex and polyhedral in the (μ, σ) parameterization. Exact representations are derived for its projection onto the scale and location coordinates, its piecewise-linear boundary, and its area. When the band limits are monotone in the order statistic index, the complete boundary can be constructed by a monotone-slope envelope algorithm in linear time after sorting the sample.

The framework permits different distribution-free CDF bands to be compared according to their induced parametric uncertainty rather than according to CDF-band width alone. The study investigates Kolmogorov–Smirnov, Dvoretzky–Kiefer–Wolfowitz, Berk–Jones/Owen, and Dümbgen–Wellner bands. Preliminary Monte Carlo experiments reveal a strong target-dependent pattern. For the normal location–scale family, the Kolmogorov–Smirnov band gives the shortest marginal interval for location, Berk–Jones gives the shortest interval for scale, and the Dümbgen–Wellner band gives the smallest expected joint area across a range of sample sizes. Experiments with Laplace and Student- t families show that the magnitude, and in some cases the ranking, of these efficiency differences depends on the base location–scale family. These observations motivate the broader problem of designing distribution-free CDF bands specifically to optimize induced parametric confidence regions.

Research provenance, attribution, and verification

Origin of the investigation

This paper has an unusual research provenance. The project began with a question posed by the human participant in a ChatGPT conversation: whether confidence information for a normal cumulative distribution function, viewed relative to an empirical CDF, could be inverted to obtain confidence information for the normal location and scale parameters.

ChatGPT initially recognized the question as one of test inversion and related it to distribution-free goodness-of-fit procedures. As the dialogue continued, the model observed that inversion of suitable order-statistic constraints produces affine inequalities in location and scale. It then generalized this observation from the normal model and the Kolmogorov–Smirnov procedure to arbitrary continuous location–scale families and, subsequently, to arbitrary simultaneous distribution-free order-statistic CDF bands.

The human participant did not prescribe this mathematical development in advance. His role during the research dialogue was principally to pose questions, choose among directions suggested by the model, request deeper investigation of particular possibilities, execute proposed computations, return numerical results, and decide which developments were worth pursuing.

Development of the research

The scientific direction changed repeatedly during the conversation. Early candidate contributions were subjected to literature searches performed by ChatGPT. In particular, the model initially

identified convexity of the Kolmogorov–Smirnov-inverted location–scale region as potentially interesting, but subsequent literature exploration showed that this property and closely related constructions were already known. The research direction was therefore narrowed rather than retaining the earlier novelty claim.

A further literature search identified recent work on inversion of distribution-free CDF bands through parametric quantile functions. This again changed the scope of the project. The investigation moved toward the special geometric structure available for two-parameter location–scale families: affine constraints, exact projections, piecewise-linear boundaries, exact region area, location–scale equivariance, and efficient envelope algorithms. The dialogue then developed the comparison of different CDF bands according to the parametric confidence regions that they induce.

ChatGPT subsequently generated the principal mathematical derivations, the envelope-based computational construction, drafts of the R implementation, the Monte Carlo design, interpretations of the simulation results, and the initial version of the present manuscript. The research record also contains corrections and changes of direction made during this process. It is therefore preserved as a chronological record rather than being rewritten as a retrospective account in which the final argument appears to have been known from the beginning.

Human verification

After the AI-generated development of the work, the human curator personally reviewed and verified the mathematical arguments and implementation before deciding to disseminate the research.

The accompanying computational materials additionally contain a separate verification suite. Among other checks, it compares the linear-time construction with an independent quadratic pairwise formula, compares the computed envelopes with direct evaluations, checks exact area calculations against numerical integration, tests affine equivariance, exercises empty and unbounded cases, verifies that band inversion reproduces the originating test event, checks seeded calibration reproducibility, and includes an empirical diagnostic for the expected computational scaling.

These automated checks are part of the reproducibility record. They are distinct from the human curator’s responsibility for examining the mathematics and implementation before publication.

Attribution

The division of contributions in this project does not fit ordinary single-author terminology. The substantive mathematical and computational research was generated predominantly by ChatGPT, while the human participant initiated the investigation, made choices about its direction, executed experiments, verified the resulting work, preserved the record, and arranged its dissemination.

For that reason, Roman Guchenko is identified on the title page as *human initiator, curator, and verifier*, rather than being presented without qualification as the scientific author of the results. ChatGPT is explicitly credited with the substantive scientific development and initial manuscript generation.

This attribution is intended to describe the provenance of the work as accurately as possible. It does not imply that the research was produced autonomously without human interaction: the complete record shows an iterative dialogue in which the human participant repeatedly selected questions and supplied computational feedback. Conversely, the human role in that dialogue should not be interpreted as a claim that the principal mathematical constructions, derivations, algorithms, or initial manuscript were originated by the human participant.

Public research record

The intended public research record consists of the following components:

- the present technical manuscript and its L^AT_EX source;
- a reader-oriented rendering of the chronological human–AI research transcript;
- an archival version of the retrieved transcript;
- the R reference implementation;
- the separate R verification suite;
- the computational notebook and its rendered form; and
- the information required to reproduce the reported simulations, including seeds and calibration settings.

The transcript is provided not merely as supplementary commentary. Because the unusual provenance of the scientific work is itself an important claim of this publication, the dialogue constitutes primary evidence for how the research developed.

At the time this version was prepared, the retrieved conversation record contained one explicitly identified truncation caused by a source-interface length limit. If the corresponding passage can be recovered from the original conversation export, it should replace the truncated passage in the archival record. Otherwise, that limitation will remain explicitly documented rather than being silently repaired.

1 Introduction

A simultaneous confidence band for a cumulative distribution function (CDF) is usually viewed as a nonparametric inferential object. Given an empirical distribution function F_n , a band provides a set of distribution functions that are compatible with the observed sample at a prescribed confidence level. If a parametric family $\{F_\theta : \theta \in \Theta\}$ is of interest, however, one may instead ask which members of this parametric family are compatible with the same band. This leads naturally to the inverted set

$$\mathcal{C}(X) = \{\theta \in \Theta : F_\theta \text{ lies in the simultaneous CDF band}\}.$$

The resulting set is a confidence region for the parameter.

Inference obtained by inversion of goodness-of-fit procedures is classical. Easterling [6] explicitly proposed parameter estimation through inversion of goodness-of-fit tests, while Littell and Rao [9] developed confidence regions for location and scale parameters based on the Kolmogorov–Smirnov (KS) statistic. Salvia [12] subsequently established important geometric properties of KS consonance sets, including convexity and finiteness for location–scale models. Thus, neither goodness-of-fit test inversion nor convexity of the KS location–scale region is new.

At the same time, the literature on distribution-free CDF bands has developed substantially beyond KS. Berk and Jones [2] introduced likelihood-ratio-type goodness-of-fit statistics with strong tail sensitivity. Owen [11] inverted the Berk–Jones construction to obtain nonparametric likelihood confidence bands, which are narrower in the tails and wider in the center than KS bands. Frey [7] studied distribution-free bands chosen directly to optimize narrowness criteria rather than test power. More recently, Dümbgen and Wellner [5] introduced a multiscale refinement of the Berk–Jones/Owen framework that preserves strong tail behavior while improving performance in the central region.

There has also been renewed interest in direct inversion of CDF bands through parametric quantile functions. Chattopadhyay, Sarkar, and Kuchibhotla [3] develop a general framework for quantile-parametrized families, motivated particularly by models in which the quantile function is tractable while the likelihood or CDF is not.

The present work focuses on a special case that has unusually strong geometric structure: two-parameter location–scale families. The quantile function is affine in the parameters,

$$Q_{\mu,\sigma}(p) = \mu + \sigma G^{-1}(p),$$

and therefore inversion of any simultaneous order-statistic CDF band produces linear inequalities in (μ, σ) . This observation allows the induced confidence set to be treated as a problem in convex geometry.

The objectives of this paper are fourfold.

First, the paper formulates CDF-band inversion for a generic continuous location–scale family and shows that the resulting confidence region is polyhedral in (μ, σ) .

Second, exact computational representations are derived for the joint region, its marginal confidence intervals for μ and σ , and its area. These calculations reduce to piecewise-linear envelope operations.

Third, this geometry is used to compare several classical and modern distribution-free confidence bands according to three parametric criteria:

$$\text{Area}(\mathcal{C}), \quad |\mathcal{C}_\mu|, \quad |\mathcal{C}_\sigma|.$$

Fourth, the study investigates how these comparisons change with the base location–scale family. The preliminary results suggest that CDF-band quality is inherently target dependent: a band that is highly efficient for scale need not be efficient for location, and a band that minimizes joint area need not minimize either marginal length.

2 General band inversion

2.1 Order-statistic representation

Let

$$X_1, \dots, X_n$$

be independent observations from a continuous distribution F , and let

$$X_{(1)} \leq \dots \leq X_{(n)}$$

denote the order statistics.

Suppose that a simultaneous distribution-free CDF band implies deterministic bounds

$$\ell_i \leq F(X_{(i)}) \leq u_i, \quad i = 1, \dots, n, \tag{1}$$

such that

$$\mathbb{P}_F(\ell_i \leq F(X_{(i)}) \leq u_i \text{ for every } i) \geq 1 - \alpha \tag{2}$$

for every continuous F .

The distribution-free nature of (2) follows from the probability integral transform. If

$$U_i = F(X_i),$$

then U_i are independent $\text{Unif}(0, 1)$ random variables and

$$F(X_{(i)}) = U_{(i)}.$$

Thus calibration of the band depends only on uniform order statistics and not on the eventual parametric family.

A small indexing detail is important. If a CDF band is originally specified on the intervals between empirical jumps, then at $X_{(i)}$ continuity of F combines information from the empirical CDF immediately before and immediately after the jump. In particular, for constant-width bands the lower endpoint involves i/n whereas the upper endpoint involves $(i-1)/n$. This distinction is essential for obtaining the intended finite-sample coverage.

2.2 Location–scale families

Consider the location–scale family

$$F_{\mu,\sigma}(x) = G\left(\frac{x-\mu}{\sigma}\right), \quad \mu \in \mathbb{R}, \quad \sigma > 0, \quad (3)$$

where G is a known continuous strictly increasing CDF.

Let

$$g(p) = G^{-1}(p),$$

and define

$$a_i = g(\ell_i), \quad b_i = g(u_i). \quad (4)$$

Values $g(0) = -\infty$ and $g(1) = +\infty$ are interpreted in the usual extended-real sense.

For a candidate (μ, σ) , condition (1) becomes

$$\ell_i \leq G\left(\frac{X_{(i)} - \mu}{\sigma}\right) \leq u_i.$$

Since G is increasing,

$$a_i \leq \frac{X_{(i)} - \mu}{\sigma} \leq b_i,$$

and therefore

$$X_{(i)} - \sigma b_i \leq \mu \leq X_{(i)} - \sigma a_i. \quad (5)$$

This gives the fundamental geometric representation.

Proposition 1 (Band-induced location–scale region). *Let (2) hold for a simultaneous order-statistic band. For the location–scale family (3), define*

$$\mathcal{C} = \bigcap_{i=1}^n \{(\mu, \sigma) : X_{(i)} - \sigma b_i \leq \mu \leq X_{(i)} - \sigma a_i\} \cap \{\sigma > 0\}. \quad (6)$$

Then $\mathcal{C}$ is a convex polyhedral set in the (μ, σ) parameterization, and

$$\mathbb{P}_{\mu_0, \sigma_0}((\mu_0, \sigma_0) \in \mathcal{C}) \geq 1 - \alpha.$$

If the order-statistic band is calibrated with exact coverage $1 - \alpha$, the induced region has the same coverage.

Proof. Each inequality in (5) defines a half-plane in (μ, σ) , and $\{\sigma > 0\}$ is convex. Their intersection is therefore convex and polyhedral.

For the true parameter (μ_0, σ_0) , membership in $\mathcal{C}$ is equivalent to

$$\ell_i \leq F_{\mu_0, \sigma_0}(X_{(i)}) \leq u_i$$

for every i . The coverage statement therefore follows directly from (2). $\square$

Remark 1. *Convexity here is naturally expressed in (μ, σ) coordinates. The transformation from scale to variance, $v = \sigma^2$, is nonlinear and does not in general preserve convexity.*

3 Exact geometry of the confidence region

Write

$$x_i = X_{(i)}$$

for convenience.

3.1 Lower and upper envelopes

For any fixed scale $s > 0$, define

$$L(s) = \max_{1 \leq i \leq n} \{x_i - b_i s\}, \quad (7)$$

$$U(s) = \min_{1 \leq i \leq n} \{x_i - a_i s\}. \quad (8)$$

Constraints involving infinite a_i or b_i are omitted whenever they are nonbinding.

The joint region can be written exactly as

$$\mathcal{C} = \{(\mu, s) : s > 0, \quad L(s) \leq \mu \leq U(s)\}. \quad (9)$$

Because L is a maximum of affine functions, it is convex and piecewise linear. Because U is a minimum of affine functions, it is concave and piecewise linear. Consequently

$$H(s) = U(s) - L(s)$$

is concave and piecewise linear.

The feasible scale values satisfy

$$H(s) \geq 0.$$

A superlevel set of a concave function on the real line is an interval. Thus, whenever the joint confidence region is nonempty and bounded in scale,

$$\mathcal{C}_\sigma = [\sigma_L, \sigma_U]. \quad (10)$$

3.2 Pairwise formula for the scale interval

For a fixed s , feasibility requires every lower constraint to lie below every upper constraint:

$$x_i - sb_i \leq x_j - sa_j \quad \text{for all } i, j.$$

Equivalently,

$$s(a_j - b_i) \leq x_j - x_i. \quad (11)$$

Let

$$d_{ij} = a_j - b_i, \quad \Delta_{ij} = x_j - x_i.$$

If $d_{ij} > 0$, (11) gives

$$s \leq \frac{\Delta_{ij}}{d_{ij}}.$$

If $d_{ij} < 0$, it gives

$$s \geq \frac{\Delta_{ij}}{d_{ij}}.$$

Hence an exact representation is

$$\sigma_L = \max \left\{ 0, \max_{d_{ij} < 0} \frac{x_j - x_i}{a_j - b_i} \right\}, \quad (12)$$

and

$$\sigma_U = \min_{d_{ij} > 0} \frac{x_j - x_i}{a_j - b_i}, \quad (13)$$

with the usual conventions if one of the index sets is empty.

This representation shows that critical scale values are ratios of observed order-statistic spacings to theoretical quantile spacings.

3.3 Marginal interval for location

The marginal confidence interval for μ is the projection of (9) onto the location coordinate:

$$\mathcal{C}_\mu = [\mu_L, \mu_U], \quad (14)$$

where

$$\mu_L = \min_{\sigma_L \leq s \leq \sigma_U} L(s), \quad (15)$$

$$\mu_U = \max_{\sigma_L \leq s \leq \sigma_U} U(s). \quad (16)$$

Since L is convex and piecewise linear, its minimum occurs at an endpoint or a breakpoint of its upper envelope. Similarly, since U is concave and piecewise linear, its maximum occurs at an endpoint or breakpoint of its lower envelope. The marginal interval can therefore be obtained exactly from finitely many line intersections, without numerical optimization over a dense parameter grid.

3.4 Area of the joint region

The area of the joint region is

$$A(\mathcal{C}) = \int_{\sigma_L}^{\sigma_U} [U(s) - L(s)] ds. \quad (17)$$

Let

$$\sigma_L = s_0 < s_1 < \cdots < s_K = \sigma_U$$

be the merged set of breakpoints of L and U . On each interval $[s_k, s_{k+1}]$,

$$L(s) = c_k + d_k s, \quad U(s) = e_k + f_k s.$$

Therefore

$$A(\mathcal{C}) = \sum_{k=0}^{K-1} \left[(e_k - c_k)(s_{k+1} - s_k) + \frac{f_k - d_k}{2} (s_{k+1}^2 - s_k^2) \right]. \quad (18)$$

Equivalently, because the gap $U - L$ is affine between consecutive breakpoints, trapezoidal integration on the breakpoint grid is exact.

3.5 Vertices

Boundary lines have the form

$$\mu = x_i - b_i \sigma$$

or

$$\mu = x_j - a_j \sigma.$$

A lower and upper boundary line intersect when

$$x_i - b_i \sigma = x_j - a_j \sigma,$$

giving

$$\sigma_{ij} = \frac{x_j - x_i}{a_j - b_i}, \quad \mu_{ij} = x_i - b_i \sigma_{ij}. \quad (19)$$

Candidate vertices are retained only if they satisfy all constraints.

4 Efficient envelope construction

The naive pairwise formulas involve $O(n^2)$ candidate intersections. The order-statistic structure permits a more efficient construction for the common case in which the confidence-band limits are monotone.

Suppose

$$\ell_1 \leq \dots \leq \ell_n, \quad u_1 \leq \dots \leq u_n.$$

Then, because $g = G^{-1}$ is increasing,

$$a_1 \leq \dots \leq a_n, \quad b_1 \leq \dots \leq b_n.$$

The lines defining L are

$$L_i(s) = x_i - b_i s,$$

whose slopes $-b_i$ are monotone. The lines defining $-U$ are

$$-U_i(s) = -x_i + a_i s,$$

whose slopes a_i are also monotone.

Proposition 2 (Monotone envelope algorithm). *Assume the band limits are monotone in i . Given the sorted sample, the active lines of L and U , together with all their breakpoints, can be constructed in $O(n)$ time.*

Proof. Consider generic lines

$$f_j(t) = c_j + m_j t$$

arriving in strictly increasing slope order. Suppose the current upper envelope contains active lines

$$h_1, \dots, h_r$$

with increasing activation points

$$z_1 < \dots < z_r.$$

A new line q has slope larger than that of h_r . Let t^* be their intersection.

If

$$t^* > z_r,$$

then h_r remains active until t^* and q becomes active thereafter.

If instead

$$t^* \leq z_r,$$

then h_r can never be part of the new envelope. Before z_r it was not part of the old envelope, while after z_r the higher-slope line q is already above it. Hence h_r may be removed permanently. The same comparison is repeated with the new last active line.

Each line is pushed onto the envelope stack once and can be popped at most once. The total number of stack operations is therefore linear in n . Equal-slope lines are handled by retaining only the line with the largest intercept.

For L , reverse the order of the lines if necessary so that the slopes $-b_i$ are increasing. For U , construct the upper envelope of $-U_i(s) = -x_i + a_i s$ and negate it. $\square$

After the two envelopes have been constructed, their sorted breakpoint lists can be merged in another linear pass. This yields $\sigma_L, \sigma_U, \mu_L, \mu_U$, the polygon vertices, and the exact area.

More explicitly, the merged pass evaluates

$$H(s) = U(s) - L(s)$$

at every breakpoint. Because H is concave and affine between successive breakpoints, its first and last zero crossings give σ_L and σ_U exactly. No pairwise intersection matrix is formed.

The reference implementation uses preallocated envelope stacks and advances active-line indices monotonically during evaluation. Thus the implementation, not only the abstract hull algorithm, has linear time and storage after the observations and slopes are ordered. The pedagogical implementation assumes, as do the bands considered here, that the probability limits are monotone in the order-statistic index. A package implementation could check this condition and sort arbitrary slopes as a fallback, at a cost of $O(n \log n)$.

For numerical stability, the implementation first centers the ordered sample at an observed median and divides it by its maximum absolute centered value. The envelope construction and all tolerance decisions are made in these normalized coordinates, using relative comparisons; intervals, boundaries, lengths, and areas are then transformed back to the original units. This preserves the affine equivariance described below even under very small or very large changes of measurement units.

Thus, conditional on already sorted observations, the complete confidence-region calculation can be performed in $O(n)$ time. Including sorting of the raw sample gives an overall complexity of $O(n \log n)$.

5 Location–scale equivariance

The geometry has a useful invariance property.

Proposition 3 (Equivariance). *Suppose*

$$X_i = \mu_0 + \sigma_0 Z_i, \quad Z_i \sim G,$$

with $\sigma_0 > 0$. Let $\mathcal{C}_X$ be the confidence region obtained from the sample X , and let $\mathcal{C}_Z$ be the corresponding region obtained from the standardized sample Z . Then

$$\mathcal{C}_X = \{(\mu_0 + \sigma_0 m, \sigma_0 s) : (m, s) \in \mathcal{C}_Z\}. \quad (20)$$

Consequently,

$$\text{Area}(\mathcal{C}_X) = \sigma_0^2 \text{Area}(\mathcal{C}_Z), \quad (21)$$

$$|\mathcal{C}_{\mu, X}| = \sigma_0 |\mathcal{C}_{\mu, Z}|, \quad (22)$$

$$|\mathcal{C}_{\sigma, X}| = \sigma_0 |\mathcal{C}_{\sigma, Z}|. \quad (23)$$

Proof. Substitute

$$x_i = \mu_0 + \sigma_0 z_i, \quad \mu = \mu_0 + \sigma_0 m, \quad \sigma = \sigma_0 s$$

into (5). After cancellation and division by σ_0 ,

$$z_i - sb_i \leq m \leq z_i - sa_i,$$

which is precisely the standardized confidence region. The scaling statements follow from the affine transformation and its Jacobian. $\square$

This result has an important computational implication: for a given base family G , sample size n , confidence level, and CDF band, the distribution of the normalized confidence-region geometry can be studied entirely under

$$\mu_0 = 0, \quad \sigma_0 = 1.$$

6 Distribution-free bands considered

Four simultaneous distribution-free CDF bands are compared. In each case the band is calibrated on the uniform probability scale and then converted to order-statistic bounds (ℓ_i, u_i) .

6.1 Kolmogorov–Smirnov

Let

$$D_n = \sup_x |F_n(x) - F(x)|.$$

Let $d_{n,\alpha}$ satisfy

$$\mathbb{P}(D_n \leq d_{n,\alpha}) = 1 - \alpha$$

for continuous F . It may be evaluated exactly or calibrated by simulation from independent uniform observations.

Using

$$D_n^+ = \max_i \left(\frac{i}{n} - U_{(i)} \right),$$

and

$$D_n^- = \max_i \left(U_{(i)} - \frac{i-1}{n} \right),$$

the order-statistic bounds are

$$\ell_i^{\text{KS}} = \max \left\{ 0, \frac{i}{n} - d_{n,\alpha} \right\}, \tag{24}$$

$$u_i^{\text{KS}} = \min \left\{ 1, \frac{i-1}{n} + d_{n,\alpha} \right\}. \tag{25}$$

6.2 Dvoretzky–Kiefer–Wolfowitz

The Dvoretzky–Kiefer–Wolfowitz inequality, with Massart’s sharp constant [4, 10], gives

$$\mathbb{P} \left(\sup_x |F_n(x) - F(x)| > \epsilon \right) \leq 2e^{-2n\epsilon^2}.$$

Taking

$$\epsilon_{n,\alpha} = \sqrt{\frac{\log(2/\alpha)}{2n}}, \tag{26}$$

gives

$$\ell_i^{\text{DKW}} = \max \left\{ 0, \frac{i}{n} - \epsilon_{n,\alpha} \right\}, \quad (27)$$

$$u_i^{\text{DKW}} = \min \left\{ 1, \frac{i-1}{n} + \epsilon_{n,\alpha} \right\}. \quad (28)$$

Unlike exactly calibrated KS, DKW is an inequality-based guarantee and is generally conservative in finite samples.

6.3 Berk–Jones/Owen

Define the Bernoulli Kullback–Leibler divergence

$$K(p, q) = p \log \frac{p}{q} + (1-p) \log \frac{1-p}{1-q}, \quad (29)$$

with the usual endpoint conventions.

For the two-sided Berk–Jones statistic,

$$T_n^{\text{BJ}} = \max_{1 \leq i \leq n} \max \left\{ nK\left(\frac{i}{n}, U_{(i)}\right), nK\left(\frac{i-1}{n}, U_{(i)}\right) \right\}. \quad (30)$$

Let $\kappa_{n,\alpha}^{\text{BJ}}$ denote its $(1-\alpha)$ quantile.

The lower order-statistic endpoint is the lower solution of

$$nK\left(\frac{i}{n}, q\right) = \kappa_{n,\alpha}^{\text{BJ}}, \quad q \leq \frac{i}{n}, \quad (31)$$

whereas the upper endpoint is the upper solution of

$$nK\left(\frac{i-1}{n}, q\right) = \kappa_{n,\alpha}^{\text{BJ}}, \quad q \geq \frac{i-1}{n}. \quad (32)$$

The scalar equations can be solved accurately by bisection or Brent’s method.

Owen [11] developed finite-sample nonparametric likelihood confidence bands by inversion of the Berk–Jones construction.

6.4 Dümbgen–Wellner

The $s = 1$, $\nu = 1$ version of Dümbgen and Wellner is used [5]. Define

$$C(t) = \log \log \left(\frac{e}{4t(1-t)} \right), \quad (33)$$

$$D(t) = \log (1 + C(t)^2), \quad (34)$$

$$C_\nu(t) = C(t) + \nu D(t). \quad (35)$$

For $p, q \in [0, 1]$, let

$$C_\nu(p, q) = \min_{\min(p,q) \leq t \leq \max(p,q)} C_\nu(t). \quad (36)$$

Define

$$H_{n,\nu}(p, q) = nK(p, q) - C_\nu(p, q). \quad (37)$$

The finite-sample statistic can be evaluated at the empirical jumps:

$$T_{n,\nu}^{\text{DW}} = \max_{1 \leq i \leq n} \max \left\{ H_{n,\nu}\left(\frac{i}{n}, U_{(i)}\right), H_{n,\nu}\left(\frac{i-1}{n}, U_{(i)}\right) \right\}. \quad (38)$$

Let $\kappa_{n,\nu,\alpha}^{\text{DW}}$ be its $(1 - \alpha)$ quantile under the uniform model.

The lower endpoint ℓ_i^{DW} is obtained by solving

$$H_{n,\nu}\left(\frac{i}{n}, q\right) = \kappa_{n,\nu,\alpha}^{\text{DW}} \quad (39)$$

on the lower side, and the upper endpoint u_i^{DW} is obtained from

$$H_{n,\nu}\left(\frac{i-1}{n}, q\right) = \kappa_{n,\nu,\alpha}^{\text{DW}} \quad (40)$$

on the upper side.

Dümbgen and Wellner show that their construction retains the strong tail performance of Berk–Jones-type bands while improving accuracy in the central region.

7 Simulation study

7.1 Design

The preliminary simulation study compares the four bands according to

$$R_A = \mathbb{E}[\text{Area}(\mathcal{C})], \quad (41)$$

$$R_\mu = \mathbb{E}[\mu_U - \mu_L], \quad (42)$$

$$R_\sigma = \mathbb{E}[\sigma_U - \sigma_L]. \quad (43)$$

Empirical joint coverage is also recorded.

The implementation defines these summaries for every replication. An empty confidence set contributes zero area and zero marginal lengths. A nonempty unbounded set contributes an infinite value for each genuinely infinite size measure. Coverage is evaluated independently of these size calculations, directly from the event

$$\ell_i \leq U_{(i)} \leq u_i \quad (i = 1, \dots, n),$$

and is therefore meaningful for empty, bounded, and unbounded sets. No missing values are discarded when Monte Carlo means are computed: an NA propagates to the corresponding reported summary, while the numbers of empty, bounded, and unbounded regions are reported separately.

For KS, Berk–Jones, and Dümbgen–Wellner, critical values were calibrated using 100,000 samples from $\text{Unif}(0, 1)$. DKW uses its analytic critical width. For each outer simulation experiment, 10,000 samples were generated.

The Monte Carlo critical values use an exchangeable-rank rule rather than an interpolated sample quantile. If $T_1, \dots, T_B$ are the calibration statistics, set

$$k = \lceil (B + 1)(1 - \alpha) \rceil$$

and use the k th order statistic as the critical value (or $+\infty$ when $k > B$). For a continuous statistic and an independent future value T_{B+1} , exchangeability gives

$$\mathbb{P}\{T_{B+1} \leq T_{(k)}\} = \frac{k}{B + 1} \geq 1 - \alpha.$$

Thus the finite calibration sample has an explicit marginal coverage guarantee. Reported outer Monte Carlo standard errors, including the binomial standard error for empirical coverage, are conditional on the realized calibrated thresholds.

Because of Proposition 3, all data were generated under location zero and scale one. More specifically, for a selected base distribution G , the simulations generated

$$U_i \sim \text{Unif}(0, 1), \quad X_i = G^{-1}(U_i).$$

This implementation also provides a useful validation: the coverage event is distribution-free and therefore should not change with G , although area and marginal lengths may change substantially.

The first experiments use the normal family. Additional experiments use the Laplace family and the Student- t_2 location-scale family. For Student- t_2 , σ denotes the scale parameter, not the standard deviation; indeed, the variance of the t_2 distribution is infinite.

7.2 Normal location-scale family

Table 1 reports preliminary results for the normal location-scale family.

Table 1: Preliminary Monte Carlo results for the normal location-scale family. Each row is based on 10,000 outer simulations.

n	Band	Expected area	Expected μ length	Expected σ length	Coverage
20	KS	2.0533	1.2553	2.7715	0.9538
20	DKW	2.3012	1.3137	2.9678	0.9649
20	Berk-Jones	1.3554	1.3669	1.7189	0.9511
20	Dümbgen-Wellner	1.3511	1.3260	1.7609	0.9537
50	KS	0.6262	0.7517	1.4084	0.9498
50	DKW	0.6721	0.7765	1.4612	0.9583
50	Berk-Jones	0.4328	0.8522	0.8776	0.9540
50	Dümbgen-Wellner	0.4291	0.8029	0.9171	0.9528
100	KS	0.2975	0.5298	0.9488	0.9515
100	DKW	0.3062	0.5371	0.9626	0.9548
100	Berk-Jones	0.2036	0.6016	0.5848	0.9488
100	Dümbgen-Wellner	0.2030	0.5613	0.6202	0.9502
200	KS	0.1430	0.3709	0.6509	0.9506
200	DKW	0.1460	0.3746	0.6573	0.9527
200	Berk-Jones	0.1004	0.4293	0.4041	0.9472
200	Dümbgen-Wellner	0.0988	0.3942	0.4292	0.9504

Three patterns are striking.

First, KS gives the smallest expected marginal interval for μ at every sample size considered.

Second, Berk-Jones gives the smallest expected marginal interval for σ . Its scale interval is approximately 38% shorter than the KS interval throughout the range $n = 20$ to 200.

Third, Dümbgen-Wellner gives the smallest expected joint area at every sample size considered. Relative to KS, the reduction in expected area is approximately 34% for $n = 20$ and remains approximately 31% for $n = 50, 100, 200$.

Dümbgen-Wellner appears to provide a compromise between the location and scale criteria. Its expected μ interval is only about 6% longer than KS over most of the range considered, while its joint area and scale interval are substantially smaller.

DKW behaves similarly to KS but is systematically more conservative, as expected from its inequality-based calibration.

7.3 Laplace and heavy-tailed Student families

Table 2 reports preliminary results for the Laplace and Student- t_2 families at $n = 50$.

Table 2: Preliminary Monte Carlo results for Laplace and Student- t_2 location-scale families, $n = 50$.

Family	Band	Expected area	Expected μ length	Expected σ length	Coverage
Laplace	KS	0.9403	0.7712	1.9236	0.9549
Laplace	DKW	0.9990	0.7924	1.9865	0.9610
Laplace	Berk-Jones	0.7878	0.9427	1.2866	0.9554
Laplace	Dümbgen-Wellner	0.7535	0.8747	1.3332	0.9534
Student- t_2	KS	1.0706	0.9335	1.8749	0.9549
Student- t_2	DKW	1.1361	0.9583	1.9355	0.9610
Student- t_2	Berk-Jones	1.0823	1.1456	1.5146	0.9554
Student- t_2	Dümbgen-Wellner	0.9880	1.0611	1.5060	0.9534

For the Laplace family, the qualitative normal-theory ranking remains: KS is best for location, Berk-Jones is best for scale, and Dümbgen-Wellner is best for joint area. However, the magnitude of the joint-area improvement is smaller. The expected area of the Dümbgen-Wellner region is about 20% below that of KS, compared with approximately 31–34% under the normal family.

The Student- t_2 results show a more substantial change. KS remains best for location and Dümbgen-Wellner remains best for joint area, but Dümbgen-Wellner now also slightly outperforms Berk-Jones in expected scale length. Moreover, the Berk-Jones expected joint area is slightly larger than the KS area.

Thus the choice of a favorable CDF band for parametric inference depends not only on the target criterion but also on the quantile geometry of the base family G .

8 Why the criteria disagree

The different behavior of the marginal location and scale intervals has a simple interpretation through the quantile function

$$Q_{\mu,\sigma}(p) = \mu + \sigma g(p).$$

Its parameter derivatives are

$$\frac{\partial Q_{\mu,\sigma}(p)}{\partial \mu} = 1, \quad \frac{\partial Q_{\mu,\sigma}(p)}{\partial \sigma} = g(p). \quad (44)$$

Location shifts all quantiles equally. Scale, in contrast, is identified through the spread of theoretical quantiles, and the magnitude of $g(p)$ often becomes large toward the tails.

This helps explain why tail-adaptive confidence bands can improve scale inference dramatically. KS and DKW allocate approximately constant vertical CDF uncertainty across the probability scale. Berk-Jones and Dümbgen-Wellner redistribute uncertainty, providing substantially tighter tail bounds. Those tail constraints can have high leverage on σ after transformation through G^{-1} .

The same observation also explains why a band optimized for scale need not be optimal for location. Allocating more of the simultaneous-error budget toward the center can improve location inference, whereas tighter tail bounds can improve scale inference.

For very heavy-tailed G , however, $G^{-1}(p)$ grows extremely rapidly as p approaches zero or one. Small changes in probability-scale band endpoints may therefore be magnified strongly in parameter space. The Student- t_2 experiment suggests that this interaction can alter the ranking of the bands themselves.

9 Relation to existing literature and proposed contribution

The main ingredients of this framework have important precedents.

Goodness-of-fit inversion as an estimation principle is classical [6]. KS-based location–scale confidence regions were developed by Littell and Rao [9], and Salvia [12] proved convexity and finiteness of KS consonance sets. Therefore, the fact that a KS-inverted location–scale region is convex cannot be regarded as a new result.

Similarly, the CDF bands used here are existing constructions. Berk–Jones [2], Owen [11], and Dümbgen–Wellner [5] provide the principal tail-adaptive procedures considered in this paper. Frey [7] studies optimal distribution-free CDF bands under band-narrowness criteria. Frey, Marrero, and Norton [8] study minimum-area confidence sets for a normal distribution. Most recently, Chattopadhyay, Sarkar, and Kuchibhotla [3] develop general inversion of distribution-free CDF bands through parametric quantile functions.

The contribution pursued here is therefore more specific:

- (i) to emphasize and exploit the fact that *arbitrary* simultaneous order-statistic CDF bands induce affine constraints in two-parameter location–scale families;
- (ii) to give an exact computational treatment of the resulting polyhedral confidence region, including marginal projections and area;
- (iii) to exploit monotonicity of standard confidence bands for efficient envelope construction;
- (iv) to compare different distribution-free CDF bands according to induced *parametric* criteria rather than CDF-band width or goodness-of-fit power;
- (v) to investigate systematically how the ranking of confidence bands depends on both the inferential target and the base location–scale family.

A complete literature review is still required before any stronger novelty claim is made, especially for the general polyhedral and algorithmic statements.

10 A parameter-targeted band optimization problem

The simulation results suggest a broader optimization problem.

Let $\mathcal{B}_\alpha$ denote a class of simultaneous distribution-free CDF bands with coverage at least $1 - \alpha$. Every $B \in \mathcal{B}_\alpha$ induces a confidence region

$$\mathcal{C}_B(X)$$

for (μ, σ) .

A band that is optimal as a confidence band for the entire CDF need not be optimal after transformation into parameter space. This motivates, for a fixed location–scale family G , the criterion

$$B_A^\star = \arg \min_{B \in \mathcal{B}_\alpha} \mathbb{E}_{0,1} [\text{Area}(\mathcal{C}_B(X))]. \quad (45)$$

Likewise one may define

$$B_\mu^\star = \arg \min_{B \in \mathcal{B}_\alpha} \mathbb{E}_{0,1} [|\mathcal{C}_{\mu,B}(X)|], \quad (46)$$

$$B_\sigma^\star = \arg \min_{B \in \mathcal{B}_\alpha} \mathbb{E}_{0,1} [|\mathcal{C}_{\sigma,B}(X)|]. \quad (47)$$

The preliminary simulations provide direct evidence that these three objectives need not agree:

$$B_A^\star \neq B_\mu^\star \neq B_\sigma^\star$$

within the set of bands currently compared.

This suggests that “optimal CDF band” is not an intrinsic concept once the CDF band is being used as an intermediate device for parametric inference. Optimality should instead be defined relative to the parameter target.

11 Further work

Several extensions are natural.

11.1 Larger simulation design

The first priority is a systematic experiment over

$$n \in \{20, 50, 100, 200, 500, 1000\}$$

and several base distributions, for example

$$\text{normal, logistic, Laplace, } t_2, t_3, t_5, t_{10},$$

and potentially asymmetric location–scale families.

The principal plots should display relative criteria

$$\frac{\mathbb{E}[A_B]}{\mathbb{E}[A_{KS}]}, \quad \frac{\mathbb{E}[L_{\mu,B}]}{\mathbb{E}[L_{\mu,KS}]}, \quad \frac{\mathbb{E}[L_{\sigma,B}]}{\mathbb{E}[L_{\sigma,KS}]} \quad (48)$$

as functions of sample size and tail behavior.

Because every method is applied to the same simulated sample, paired comparisons should also be reported, for example

$$\mathbb{P}(A_{DW} < A_{KS}).$$

11.2 Coverage matching

Native calibrations do not always have identical finite-sample coverage. DKW is deliberately conservative. The rank-calibrated KS, BJ, and DW thresholds have an explicit marginal coverage $k/(B+1)$, although their conditional coverage given one realized calibration sample still fluctuates around that level.

Two comparisons are therefore useful:

- (i) the procedures under their native published calibration;
- (ii) a coverage-matched comparison in which thresholds are adjusted to give a common finite-sample coverage as close as possible to $1 - \alpha$.

The second comparison isolates geometric efficiency from differences in conservativeness.

11.3 Asymptotic theory

The striking stability of some normal-theory ratios suggests that asymptotic relative efficiencies may exist. For example, the Berk–Jones scale interval was approximately 38% shorter than the KS scale interval throughout the preliminary range $n = 20$ to 200.

A theoretical question is whether quantities such as

$$\frac{\mathbb{E}[|\mathcal{C}_{\sigma, BJ}|]}{\mathbb{E}[|\mathcal{C}_{\sigma, KS}|]}$$

converge, and how the limit depends on the tail behavior of G^{-1} .

11.4 Optimization over bands

The most ambitious extension is to optimize the band limits (ℓ_i, u_i) directly subject to the distribution-free coverage constraint

$$\mathbb{P}(\ell_i \leq U_{(i)} \leq u_i \text{ for every } i) \geq 1 - \alpha.$$

Frey [7] optimizes CDF bands according to CDF-scale narrowness criteria. The present framework suggests replacing that objective by expected induced parameter-region area or marginal length. The resulting optimal band would depend on the base family G and on the inferential target.

11.5 Robustness and misspecification

A further extension is to distinguish confidence-region efficiency under the assumed model from behavior under misspecification. If data are generated from a distribution outside the chosen location–scale family, the confidence region may become empty. The event

$$\mathcal{C}(X) = \emptyset$$

then has a natural model-compatibility interpretation: no member of the specified family lies inside the chosen simultaneous CDF band.

Thus the same geometric object can potentially combine parametric uncertainty and goodness-of-fit information.

12 Conclusion

A simultaneous distribution-free CDF band can be viewed not only as a nonparametric confidence object but also as a mechanism for generating finite-sample confidence regions within a parametric family.

For a continuous two-parameter location–scale family, this inversion has a particularly simple structure. Every order-statistic band constraint becomes affine in (μ, σ) , so the joint confidence region is convex and polyhedral. Its scale projection, location projection, vertices, and area can be computed exactly using piecewise-linear envelopes rather than parameter grids or repeated simulation over candidate parameter values.

This geometry provides a common framework in which different CDF bands can be compared according to the uncertainty they induce about parameters. Preliminary results demonstrate that such comparisons are not trivial. Under normality, KS is most efficient for location, Berk–Jones is most efficient for scale, and Dümbgen–Wellner is most efficient for joint area among the bands considered. The same broad pattern appears for Laplace data, while sufficiently heavy tails alter some rankings.

The emerging conclusion is that the best distribution-free CDF band for parametric inference depends on what parameter uncertainty is being optimized and on the geometry of the underlying quantile function. This motivates the study of parameter-targeted distribution-free bands as a distinct inferential problem.

A Algorithmic summary

Given a sample $x_1, \dots, x_n$, band bounds (ℓ_i, u_i) , and a base quantile function $g = G^{-1}$:

1. Sort the data:

$$x_{(1)} \leq \dots \leq x_{(n)}.$$

2. Transform the probability limits:

$$a_i = g(\ell_i), \quad b_i = g(u_i).$$

3. Form the lower envelope

$$L(s) = \max_i \{x_{(i)} - b_i s\}.$$

4. Form the upper envelope

$$U(s) = \min_i \{x_{(i)} - a_i s\}.$$

5. Merge the two ordered breakpoint lists, evaluate

$$H(s) = U(s) - L(s),$$

and locate its first and last zero crossings. These are the endpoints of the feasible scale interval.

6. Compute

$$\mu_L = \min L(s), \quad \mu_U = \max U(s)$$

over the feasible scale interval.

7. Merge the envelope breakpoints and compute

$$\text{Area}(\mathcal{C}) = \int [U(s) - L(s)] ds$$

exactly interval by interval.

B Monte Carlo calibration

For KS, BJ, and DW, the critical values can be calibrated once for each (n, α) using independent uniform samples. These critical values and the resulting probability-scale bounds are then reused in every outer Monte Carlo replication and for every base location-scale family.

The reference code implements the exchangeable-rank choice $k = \lceil (B+1)(1-\alpha) \rceil$ directly. Each simulation function has an explicit seed argument, so a displayed result can be reproduced without adding random-number-state machinery to the main algorithm.

This separation is computationally important:

$$\text{band calibration} \longrightarrow (\ell_i, u_i) \longrightarrow \text{family-specific quantile transformation.}$$

Changing from the normal family to Laplace or Student- t , for example, requires changing only

$$G^{-1},$$

not recalibrating the distribution-free band.

C Reference R implementation

The following commented R implementation contains the complete computational pipeline used in this note: Monte Carlo calibration of the KS, Berk–Jones, and Dümmbgen–Wellner bands; construction of the DKW bounds; exact envelope-based computation of the induced location-scale confidence region; and the outer Monte Carlo comparison. The final driver accepts an arbitrary base quantile function, so the same code applies to the normal, Laplace, and Student- t experiments.

The listing is intentionally pedagogical rather than package-level production code. It assumes that arguments have the advertised types and ranges and omits routine checks for malformed inputs. It does retain the mathematical edge cases that affect the result—empty and unbounded confidence sets, infinite transformed probability endpoints, and relative numerical tolerances after affine normalization. This keeps the statistical construction visible while the companion verification script tests correctness independently.

The listing is consolidated from the accompanying notebook `location_scale_research.ipynb`. It retains the explanatory comments from the notebook. Sourcing the file only defines functions: the illustrative example and the complete reproduction study are explicit drivers and are not launched automatically. The latter calibrates each required sample size once and reuses those bands across the normal, Laplace, and Student- t_2 experiments. Notebook outputs are omitted; the numerical summaries reported in the main text remain the record of the preliminary runs.

Listing 1: Pedagogical commented R implementation and reproduction driver.

```

1  # =====
2  # PEDAGOGICAL R IMPLEMENTATION
3  #
4  # From distribution-free CDF bands to confidence regions for a
5  # location--scale family.
6  #
7  # The code assumes that inputs have the advertised form: n and B are
8  # positive integers, 0 < alpha < 1, probability bounds are ordered, and
9  # qfun is a vectorized quantile function. This keeps the implementation
10 # focused on the statistical and geometric ideas.
11 # =====
12
13
14 # =====
15 # 1. DISTRIBUTION-FREE ORDER-STATISTIC BANDS
16 # =====
17
18 # If T_1,...,T_B and a future statistic T_new are exchangeable, the kth
19 # calibration order statistic gives marginal coverage k/(B+1). Choosing
20 # k = ceiling((B+1)(1-alpha)) therefore avoids an arbitrary interpolated
21 # sample quantile.
22 mc_rank_critical_value <- function(statistics, alpha) {
23   B <- length(statistics)
24   k <- ceiling((B + 1) * (1 - alpha))
25   if (k > B) Inf else sort(statistics, partial = k)[k]
26 }
27
28
29 simulate_critical_value <- function(
30   statistic,
31   n,
32   alpha = 0.05,
33   B = 100000,
34   seed = 123,
35   ...
36 ) {
37   set.seed(seed)
38   statistics <- numeric(B)
39   for (b in seq_len(B)) {
40     statistics[b] <- statistic(runif(n), ...)
41   }
42   mc_rank_critical_value(statistics, alpha)
43 }
44
45
46 # -----
47 # Kolmogorov--Smirnov
48 # -----
49
50 ks_statistic <- function(u) {
51   u <- sort(u)
52   n <- length(u)

```

```

53   i <- seq_len(n)
54   max(i / n - u, u - (i - 1) / n)
55 }
56
57
58 ks_bounds_mc <- function(
59   n,
60   alpha = 0.05,
61   B = 100000,
62   seed = 123,
63   critical = NULL
64 ) {
65   if (is.null(critical)) {
66     critical <- simulate_critical_value(
67       ks_statistic, n, alpha, B, seed
68     )
69   }
70
71   i <- seq_len(n)
72   bounds <- data.frame(
73     i = i,
74     lower = pmax(0, i / n - critical),
75     upper = pmin(1, (i - 1) / n + critical)
76   )
77
78   list(critical_value = critical, bounds = bounds)
79 }
80
81
82 # -----
83 # Dvoretzky--Kiefer--Wolfowitz
84 # -----
85
86 dkwbounds <- function(n, alpha = 0.05) {
87   epsilon <- sqrt(log(2 / alpha) / (2 * n))
88   i <- seq_len(n)
89
90   data.frame(
91     i = i,
92     lower = pmax(0, i / n - epsilon),
93     upper = pmin(1, (i - 1) / n + epsilon)
94   )
95 }
96
97
98 # -----
99 # Berk--Jones/Owen
100 # -----
101
102 # Bernoulli Kullback--Leibler divergence, with 0 log(0/y) = 0.
103 kl_bern <- function(p, q) {
104   xlogy <- function(x, y) {
105     value <- x * log(x / y)
106     value[x == 0] <- 0
107     value
108   }
109
110   xlogy(p, q) + xlogy(1 - p, 1 - q)
111 }
112
113
114 bj_statistic <- function(u) {
115   u <- sort(u)
116   n <- length(u)
117   i <- seq_len(n)
118
119   max(
120     n * kl_bern((i - 1) / n, u),
121     n * kl_bern(i / n, u)
122   )
123 }
124
125

```

```

126 # Solve  $K(p, q) = \text{critical}/n$  on both sides of  $p$ .
127 bj_interval_at_p <- function(p, critical, n, tol = 1e-10) {
128   if (is.infinite(critical)) return(c(lower = 0, upper = 1))
129
130   target <- critical / n
131   f <- function(q) kl_bern(p, q) - target
132
133   lower <- if (p == 0) {
134     0
135   } else {
136     uniroot(f, c(.Machine$double.eps, p), tol = tol)$root
137   }
138
139   upper <- if (p == 1) {
140     1
141   } else {
142     uniroot(f, c(p, 1 - .Machine$double.eps), tol = tol)$root
143   }
144
145   c(lower = lower, upper = upper)
146 }
147
148
149 bj_bounds <- function(
150   n,
151   alpha = 0.05,
152   B = 100000,
153   seed = 123,
154   critical = NULL
155 ) {
156   if (is.null(critical)) {
157     critical <- simulate_critical_value(
158       bj_statistic, n, alpha, B, seed
159     )
160   }
161
162   i <- seq_len(n)
163   lower <- vapply(
164     i / n,
165     function(p) bj_interval_at_p(p, critical, n)["lower"],
166     numeric(1)
167   )
168   upper <- vapply(
169     (i - 1) / n,
170     function(p) bj_interval_at_p(p, critical, n)["upper"],
171     numeric(1)
172   )
173
174   list(
175     critical_value = critical,
176     bounds = data.frame(i = i, lower = lower, upper = upper)
177   )
178 }
179
180
181 # -----
182 # Duembgen--Wellner, with  $s = 1$  and  $\nu = 1$  by default
183 # -----
184
185 C_nu <- function(t, nu = 1) {
186   value <- rep(Inf, length(t))
187   interior <- t > 0 & t < 1
188
189   C <- log(log(exp(1) / (4 * t[interior] * (1 - t[interior]))))
190   value[interior] <- C + nu * log(1 + C^2)
191   value
192 }
193
194
195 # min_{v between p and q} C_nu(v). The correction decreases toward 1/2.
196 C_nu_biv <- function(p, q, nu = 1) {
197   lo <- pmin(p, q)
198   hi <- pmax(p, q)

```

```

199
200   ifelse(
201     hi < 0.5,
202     C_nu(hi, nu),
203     ifelse(lo > 0.5, C_nu(lo, nu), 0)
204   )
205 }
206
207
208 dw_discrepancy <- function(p, q, n, nu = 1) {
209   n * kl_bern(p, q) - C_nu_biv(p, q, nu)
210 }
211
212
213 dw_statistic <- function(u, nu = 1) {
214   u <- sort(u)
215   n <- length(u)
216   i <- seq_len(n)
217
218   max(
219     dw_discrepancy((i - 1) / n, u, n, nu),
220     dw_discrepancy(i / n, u, n, nu)
221   )
222 }
223
224
225 # The upper endpoint is the largest q >= p satisfying the DW inequality.
226 dw_upper_at_p <- function(p, n, critical, nu = 1, tol = 1e-10) {
227   if (p == 1 || is.infinite(critical)) return(1)
228
229   f <- function(q) dw_discrepancy(p, q, n, nu) - critical
230   q_max <- 1 - .Machine$double.eps
231
232   if (f(q_max) <= 0) 1 else uniroot(f, c(p, q_max), tol = tol)$root
233 }
234
235
236 dw_bounds <- function(
237   n,
238   alpha = 0.05,
239   B = 100000,
240   nu = 1,
241   seed = 123,
242   critical = NULL
243 ) {
244   if (is.null(critical)) {
245     critical <- simulate_critical_value(
246       dw_statistic, n, alpha, B, seed, nu = nu
247     )
248   }
249
250   i <- seq_len(n)
251   lower <- 1 - vapply(
252     1 - i / n,
253     function(p) dw_upper_at_p(p, n, critical, nu),
254     numeric(1)
255   )
256   upper <- vapply(
257     (i - 1) / n,
258     function(p) dw_upper_at_p(p, n, critical, nu),
259     numeric(1)
260   )
261
262   list(
263     critical_value = critical,
264     bounds = data.frame(i = i, lower = lower, upper = upper)
265   )
266 }
267
268
269 # Calibrate once for a given (n, alpha), then reuse these bounds for every
270 # location--scale family in the outer simulation.
271 calibrate_bands <- function(

```

```

272     n,
273     alpha = 0.05,
274     B = 100000,
275     seed = 123
276 ) {
277   KS <- ks_bounds_mc(n, alpha, B, seed + 1)
278   BJ <- bj_bounds(n, alpha, B, seed + 2)
279   DW <- dw_bounds(n, alpha, B, seed = seed + 3)
280
281   list(
282     n = n,
283     alpha = alpha,
284     B = B,
285     bounds = list(
286       KS = KS$bounds,
287       DKW = dkw_bounds(n, alpha),
288       Berk_Jones = BJ$bounds,
289       Duembgen_Wellner = DW$bounds
290     ),
291     critical_values = c(
292       KS = KS$critical_value,
293       Berk_Jones = BJ$critical_value,
294       Duembgen_Wellner = DW$critical_value
295     )
296   )
297 }
298
299
300 # =====
301 # 2. EXACT LOCATION--SCALE GEOMETRY
302 # =====
303
304 # Upper envelope of lines c_j + m_j*s. Slopes must be nondecreasing.
305 # The stack stores each surviving line and the point where it becomes active.
306 upper_envelope <- function(intercept, slope) {
307   n <- length(slope)
308
309   # Among parallel lines, only the highest intercept can be active.
310   unique_c <- numeric(n)
311   unique_m <- numeric(n)
312   n_unique <- 0L
313
314   for (j in seq_len(n)) {
315     if (n_unique > 0L && slope[j] == unique_m[n_unique]) {
316       unique_c[n_unique] <- max(unique_c[n_unique], intercept[j])
317     } else {
318       n_unique <- n_unique + 1L
319       unique_c[n_unique] <- intercept[j]
320       unique_m[n_unique] <- slope[j]
321     }
322   }
323
324   unique_c <- unique_c[seq_len(n_unique)]
325   unique_m <- unique_m[seq_len(n_unique)]
326
327   hull_c <- numeric(n_unique)
328   hull_m <- numeric(n_unique)
329   start <- numeric(n_unique)
330   size <- 0L
331
332   for (j in seq_len(n_unique)) {
333     cj <- unique_c[j]
334     mj <- unique_m[j]
335
336     while (size > 1L) {
337       crossing <- (hull_c[size] - cj) / (mj - hull_m[size])
338       if (crossing > start[size]) break
339       size <- size - 1L
340     }
341
342     size <- size + 1L
343     hull_c[size] <- cj
344     hull_m[size] <- mj

```

```

345     start[size] <- if (size == 1L) {
346       -Inf
347     } else {
348       (hull_c[size - 1L] - c_j) / (m_j - hull_m[size - 1L])
349     }
350   }
351 }
352 keep <- seq_len(size)
353 list(
354   intercept = hull_c[keep],
355   slope = hull_m[keep],
356   start = start[keep]
357 )
358 }
359
360
361 eval_envelope <- function(envelope, s) {
362   active <- findInterval(s, envelope$start)
363   active <- pmax.int(1L, pmin.int(active, length(envelope$start)))
364   envelope$intercept[active] + envelope$slope[active] * s
365 }
366
367 # Merge two already sorted breakpoint lists in linear time.
368 merge_sorted_unique <- function(x, y) {
369   x <- x[is.finite(x)]
370   y <- y[is.finite(y)]
371   answer <- numeric(length(x) + length(y))
372   i <- j <- 1L
373   k <- 0L
374
375   while (i <= length(x) || j <= length(y)) {
376     if (j > length(y) || (i <= length(x) && x[i] <= y[j])) {
377       value <- x[i]
378       i <- i + 1L
379     } else {
380       value <- y[j]
381       j <- j + 1L
382     }
383
384     if (k == 0L || value != answer[k]) {
385       k <- k + 1L
386       answer[k] <- value
387     }
388   }
389
390   if (k == 0L) numeric(0) else answer[seq_len(k)]
391 }
392
393
394 # Evaluation is also linear when both the envelope starts and s are sorted.
395 eval_envelope_sorted <- function(envelope, s) {
396   answer <- numeric(length(s))
397   active <- 1L
398
399   for (j in seq_along(s)) {
400     while (active < length(envelope$start) &&
401           envelope$start[active + 1L] <= s[j]) {
402       active <- active + 1L
403     }
404     answer[j] <- envelope$intercept[active] + envelope$slope[active] * s[j]
405   }
406
407   answer
408 }
409
410
411 # H(s) = U(s) - L(s) is concave and piecewise affine. Its nonnegative set
412 # is therefore one interval. Scan the merged envelope breakpoints and
413 # interpolate its first and last zero crossings.
414 sigma_interval_from_envelopes <- function(env_L, env_V, tol = 1e-12) {
415   starts_L <- env_L$start[is.finite(env_L$start) & env_L$start > 0]
416   starts_V <- env_V$start[is.finite(env_V$start) & env_V$start > 0]

```

```

418 knots <- c(0, merge_sorted_unique(starts_L, starts_V))
419
420 L <- eval_envelope_sorted(env_L, knots)
421 V <- eval_envelope_sorted(env_V, knots)
422 gap <- -V - L
423 gap_tol <- tol * pmax(1, abs(L), abs(V))
424 gap[abs(gap) <= gap_tol] <- 0
425
426 tail_slope <- -tail(env_V$slope, 1) - tail(env_L$slope, 1)
427 slope_tol <- tol * max(
428   1, abs(tail(env_V$slope, 1)), abs(tail(env_L$slope, 1))
429 )
430
431 feasible <- which(gap >= 0)
432
433 if (length(feasible) == 0L) {
434   if (tail_slope > slope_tol) {
435     lower <- tail(knots, 1) - tail(gap, 1) / tail_slope
436     return(c(lower = max(0, lower), upper = Inf))
437   }
438   return(c(lower = NA_real_, upper = NA_real_))
439 }
440
441 first <- feasible[1]
442 last <- tail(feasible, 1)
443
444 lower <- if (first == 1L) {
445   0
446 } else {
447   x1 <- knots[first - 1L]
448   x2 <- knots[first]
449   y1 <- gap[first - 1L]
450   y2 <- gap[first]
451   x1 - y1 * (x2 - x1) / (y2 - y1)
452 }
453
454 upper <- if (last < length(knots)) {
455   x1 <- knots[last]
456   x2 <- knots[last + 1L]
457   y1 <- gap[last]
458   y2 <- gap[last + 1L]
459   x1 - y1 * (x2 - x1) / (y2 - y1)
460 } else if (tail_slope < -slope_tol) {
461   tail(knots, 1) - tail(gap, 1) / tail_slope
462 } else {
463   Inf
464 }
465
466 c(lower = max(0, lower), upper = upper)
467 }
468
469
470 minimum_of_envelope <- function(envelope, tol = 1e-12) {
471   knots <- c(0, envelope$start[is.finite(envelope$start) & envelope$start > 0])
472   values <- eval_envelope_sorted(envelope, knots)
473   slope_tol <- tol * max(1, abs(tail(envelope$slope, 1)))
474
475   if (tail(envelope$slope, 1) < -slope_tol) -Inf else min(values)
476 }
477
478
479 empty_region <- function(a, b) {
480   list(
481     status = "empty",
482     feasible = FALSE,
483     bounded = TRUE,
484     sigma_interval = c(lower = NA_real_, upper = NA_real_),
485     sigma_length = 0,
486     mu_interval = c(lower = NA_real_, upper = NA_real_),
487     mu_length = 0,
488     area = 0,
489     a = a,
490     b = b,

```

```

491     boundary = NULL,
492     polygon = NULL
493   )
494 }
495
496
497 # Work in centered, dimensionless coordinates. This is the actual geometric
498 # construction; the public wrapper below restores the original units.
499 location_scale_confidence_normalized <- function(
500   x,
501   lower,
502   upper,
503   qfun = qnorm,
504   tol = 1e-12
505 ) {
506   n <- length(x)
507   a <- qfun(lower)
508   b <- qfun(upper)
509
510   # These transformed endpoints would require an infinite parameter value.
511   if (any(a == Inf) || any(b == -Inf)) return(empty_region(a, b))
512
513   finite_a <- is.finite(a)
514   finite_b <- is.finite(b)
515
516   env_L <- if (any(finite_b)) {
517     index <- rev(which(finite_b))
518     upper_envelope(x[index], -b[index])
519   } else NULL
520
521   env_V <- if (any(finite_a)) {
522     index <- which(finite_a)
523     upper_envelope(-x[index], a[index])
524   } else NULL
525
526   # If one side has no finite constraints, the set has infinite area.
527   if (is.null(env_L) || is.null(env_V)) {
528     mu_lower <- if (is.null(env_L)) -Inf else minimum_of_envelope(env_L, tol)
529     mu_upper <- if (is.null(env_V)) Inf else -minimum_of_envelope(env_V, tol)
530
531     return(list(
532       status = "unbounded",
533       feasible = TRUE,
534       bounded = FALSE,
535       sigma_interval = c(lower = 0, upper = Inf),
536       sigma_length = Inf,
537       mu_interval = c(lower = mu_lower, upper = mu_upper),
538       mu_length = Inf,
539       area = Inf,
540       a = a,
541       b = b,
542       boundary = NULL,
543       polygon = NULL
544     ))
545   }
546
547   sigma_interval <- sigma_interval_from_envelopes(env_L, env_V, tol)
548   if (anyNA(sigma_interval) || sigma_interval["upper"] <= 0) {
549     return(empty_region(a, b))
550   }
551
552   sigma_lower <- sigma_interval["lower"]
553   sigma_upper <- sigma_interval["upper"]
554
555   # Unbounded scale interval: retain the finite breakpoints and examine the
556   # final affine pieces to decide which projections and measures diverge.
557   if (is.infinite(sigma_upper)) {
558     starts_L <- env_L$start[is.finite(env_L$start) & env_L$start > sigma_lower]
559     starts_V <- env_V$start[is.finite(env_V$start) & env_V$start > sigma_lower]
560     knots <- c(sigma_lower, merge_sorted_unique(starts_L, starts_V))
561
562     L <- eval_envelope_sorted(env_L, knots)
563     U <- -eval_envelope_sorted(env_V, knots)

```

```

564 gap <- U - L
565 gap_tol <- tol * pmax(1, abs(L), abs(U))
566 gap[abs(gap) <= gap_tol] <- 0
567
568 gap_slope <- -tail(env_V$slope, 1) - tail(env_L$slope, 1)
569 slope_tol <- tol * max(
570   1, abs(tail(env_V$slope, 1)), abs(tail(env_L$slope, 1))
571 )
572 prefix_area <- if (length(knots) == 1L) 0 else {
573   sum(0.5 * (gap[-length(gap)] + gap[-1]) * diff(knots))
574 }
575 area <- if (gap_slope > slope_tol || tail(gap, 1) > tail(gap_tol, 1)) {
576   Inf
577 } else prefix_area
578
579 lower_slope <- tail(env_L$slope, 1)
580 upper_slope <- -tail(env_V$slope, 1)
581 mu_lower <- if (lower_slope < -slope_tol) -Inf else min(L)
582 mu_upper <- if (upper_slope > slope_tol) Inf else max(U)
583
584 return(list(
585   status = "unbounded",
586   feasible = TRUE,
587   bounded = FALSE,
588   sigma_interval = sigma_interval,
589   sigma_length = Inf,
590   mu_interval = c(lower = mu_lower, upper = mu_upper),
591   mu_length = if (is.finite(mu_lower) && is.finite(mu_upper)) {
592     max(0, mu_upper - mu_lower)
593   } else Inf,
594   area = area,
595   a = a,
596   b = b,
597   boundary = data.frame(
598     sigma = knots, lower_mu = L, upper_mu = U, width = gap
599   ),
600   polygon = NULL
601 ))
602 }
603
604 # Bounded region: every boundary is affine between consecutive knots, so
605 # trapezoidal integration is exact rather than an approximation.
606 starts_L <- env_L$start[
607   is.finite(env_L$start) & env_L$start > sigma_lower &
608   env_L$start < sigma_upper
609 ]
610 starts_V <- env_V$start[
611   is.finite(env_V$start) & env_V$start > sigma_lower &
612   env_V$start < sigma_upper
613 ]
614 knots <- c(
615   sigma_lower,
616   merge_sorted_unique(starts_L, starts_V),
617   sigma_upper
618 )
619 knots <- unique(knots)
620
621 L <- eval_envelope_sorted(env_L, knots)
622 U <- -eval_envelope_sorted(env_V, knots)
623 gap <- U - L
624 gap[gap < 0 & abs(gap) <= tol * pmax(1, abs(L), abs(U))] <- 0
625
626 area <- if (length(knots) == 1L) 0 else {
627   sum(0.5 * (gap[-length(gap)] + gap[-1]) * diff(knots))
628 }
629
630 boundary <- data.frame(
631   sigma = knots,
632   lower_mu = L,
633   upper_mu = U,
634   width = gap
635 )
636 polygon <- rbind(

```

```

637     data.frame(sigma = knots, mu = L, boundary = "lower"),
638     data.frame(sigma = rev(knots), mu = rev(U), boundary = "upper")
639 )
640
641 list(
642   status = "bounded",
643   feasible = TRUE,
644   bounded = TRUE,
645   sigma_interval = sigma_interval,
646   sigma_length = sigma_upper - sigma_lower,
647   mu_interval = c(lower = min(L), upper = max(U)),
648   mu_length = max(U) - min(L),
649   area = area,
650   a = a,
651   b = b,
652   boundary = boundary,
653   polygon = polygon
654 )
655 }
656
657
658 # Centering and scaling make numerical tolerances independent of measurement
659 # units. The final lines simply restore the original location and scale.
660 location_scale_confidence <- function(
661   x,
662   lower,
663   upper,
664   qfun = qnorm,
665   tol = 1e-12
666 ) {
667   x <- sort(x)
668   center <- x[(length(x) + 1L) %/% 2L]
669   data_scale <- max(abs(x - center))
670   if (data_scale == 0) data_scale <- 1
671
672   fit <- location_scale_confidence_normalized(
673     (x - center) / data_scale,
674     lower,
675     upper,
676     qfun,
677     tol
678   )
679
680   fit$sigma_interval <- data_scale * fit$sigma_interval
681   fit$sigma_length <- data_scale * fit$sigma_length
682   fit$mu_interval <- center + data_scale * fit$mu_interval
683   fit$mu_length <- data_scale * fit$mu_length
684   fit$area <- if (fit$area == 0) 0 else data_scale^2 * fit$area
685
686   if (!is.null(fit$boundary)) {
687     fit$boundary$sigma <- data_scale * fit$boundary$sigma
688     fit$boundary$lower_mu <- center + data_scale * fit$boundary$lower_mu
689     fit$boundary$upper_mu <- center + data_scale * fit$boundary$upper_mu
690     fit$boundary$width <- data_scale * fit$boundary$width
691   }
692   if (!is.null(fit$polygon)) {
693     fit$polygon$sigma <- data_scale * fit$polygon$sigma
694     fit$polygon$mu <- center + data_scale * fit$polygon$mu
695   }
696
697   fit
698 }
699
700
701 # =====
702 # 3. MONTE CARLO COMPARISON
703 # =====
704
705 compare_bands_mc <- function(
706   n = 50,
707   alpha = 0.05,
708   M = 10000,
709   band_calibration_B = 100000,

```

```

710     seed = 123,
711     ls.quantile.function = qnorm,
712     bands = NULL,
713     progress = interactive()
714 ) {
715     calibration <- if (is.null(bands)) {
716         calibrate_bands(n, alpha, band_calibration_B, seed)
717     } else if (!is.null(bands$bounds)) {
718         bands
719     } else NULL
720
721     if (!is.null(calibration)) bands <- calibration$bounds
722     methods <- names(bands)
723
724     area <- mu_length <- sigma_length <- matrix(
725         NA_real_, M, length(methods), dimnames = list(NULL, methods)
726     )
727     coverage <- matrix(FALSE, M, length(methods), dimnames = list(NULL, methods))
728     status <- matrix("", M, length(methods), dimnames = list(NULL, methods))
729
730     set.seed(seed)
731     for (m in seq_len(M)) {
732         # Sampling on the probability scale gives both the data and a direct
733         # check of the simultaneous band event.
734         u <- sort(runif(n))
735         x <- ls.quantile.function(u)
736
737         for (j in seq_along(methods)) {
738             band <- bands[[j]]
739             coverage[m, j] <- all(band$lower <= u & u <= band$upper)
740
741             fit <- location_scale_confidence(
742                 x, band$lower, band$upper, ls.quantile.function
743             )
744             area[m, j] <- fit$area
745             mu_length[m, j] <- fit$mu_length
746             sigma_length[m, j] <- fit$sigma_length
747             status[m, j] <- fit$status
748         }
749
750         if (progress && m %% 1000 == 0) {
751             message("Completed ", m, " of ", M)
752         }
753     }
754
755     standard_error <- function(z) {
756         if (all(is.finite(z))) sd(z) / sqrt(length(z)) else NA_real_
757     }
758
759     empirical_coverage <- colMeans(coverage)
760     summary <- data.frame(
761         method = methods,
762         expected_area = colMeans(area),
763         se_area = apply(area, 2, standard_error),
764         expected_mu_length = colMeans(mu_length),
765         se_mu_length = apply(mu_length, 2, standard_error),
766         expected_sigma_length = colMeans(sigma_length),
767         se_sigma_length = apply(sigma_length, 2, standard_error),
768         empirical_coverage = empirical_coverage,
769         se_coverage = sqrt(empirical_coverage * (1 - empirical_coverage) / M),
770         n_replications = M,
771         n_empty = colSums(status == "empty"),
772         n_bounded = colSums(status == "bounded"),
773         n_unbounded = colSums(status == "unbounded"),
774         row.names = NULL
775     )
776
777     list(
778         summary = summary,
779         area = area,
780         mu_length = mu_length,
781         sigma_length = sigma_length,
782         coverage = coverage,

```

```

783     region_status = status,
784     bands = bands,
785     band_calibration = calibration
786   )
787 }
788
789
790 q_laplace <- function(p) {
791   ifelse(p < 0.5, log(2 * p), -log(2) - log1p(-p))
792 }
793
794
795 # This function contains every experiment reported in the paper. Each sample
796 # size is calibrated once; the resulting bands are reused across families.
797 run_reproduction_study <- function(
798   alpha = 0.05,
799   M = 10000,
800   calibration_B = 100000,
801   seed = 42,
802   save_path = NULL,
803   progress = interactive()
804 ) {
805   sizes <- c(20L, 50L, 100L, 200L)
806   calibrations <- setNames(
807     lapply(
808       sizes,
809       function(n) calibrate_bands(n, alpha, calibration_B, seed + n)
810     ),
811     sizes
812   )
813
814   run_family <- function(n, qfun) {
815     compare_bands_mc(
816       n = n,
817       alpha = alpha,
818       M = M,
819       seed = seed,
820       ls.quantile.function = qfun,
821       bands = calibrations[[as.character(n)]],
822       progress = progress
823     )
824   }
825
826   results <- list(
827     normal = setNames(lapply(sizes, run_family, qfun = qnorm), sizes),
828     laplace = list('50' = run_family(50L, q_laplace)),
829     student_t2 = list('50' = run_family(50L, function(p) qt(p, df = 2)))
830   )
831
832   if (!is.null(save_path)) saveRDS(results, save_path)
833   results
834 }
835
836
837 # A compact example for the notebook. Set draw = TRUE to plot the regions.
838 run_illustrative_example <- function(
839   n = 50,
840   alpha = 0.05,
841   calibration_B = 100000,
842   seed = 1,
843   draw = interactive()
844 ) {
845   calibration <- calibrate_bands(n, alpha, calibration_B, seed)
846   set.seed(seed)
847   x <- rnorm(n, mean = 2, sd = 3)
848
849   fits <- lapply(
850     calibration$bounds,
851     function(band) {
852       location_scale_confidence(x, band$lower, band$upper, qnorm)
853     }
854   )
855 }

```

```

856 summary <- data.frame(
857   method = names(fits),
858   status = vapply(fits, '[', character(1), "status"),
859   area = vapply(fits, '[', numeric(1), "area"),
860   mu_length = vapply(fits, '[', numeric(1), "mu_length"),
861   sigma_length = vapply(fits, '[', numeric(1), "sigma_length")
862 )
863
864 if (draw) {
865   bounded <- vapply(fits, function(fit) fit$status == "bounded", logical(1))
866   old_par <- par(mfrow = grDevices::n2mfrow(sum(bounded)))
867   on.exit(par(old_par))
868
869   for (method in names(fits)[bounded]) {
870     polygon_data <- fits[[method]]$polygon
871     plot(
872       polygon_data$sigma,
873       polygon_data$mu,
874       type = "n",
875       xlab = expression(sigma),
876       ylab = expression(mu),
877       main = method
878     )
879     polygon(polygon_data$sigma, polygon_data$mu)
880   }
881 }
882
883 list(sample = x, fits = fits, summary = summary)
884 }

```

D Verification code

The separate companion file `location_scale_checks.R` checks the implementation against the independent $O(n^2)$ pairwise formula on randomized inputs, direct envelope maxima and minima, and dense numerical area integration. It also contains regression tests for one-segment and final-segment envelope evaluation; extreme affine changes of units; empty and unbounded confidence sets; equivalence between each inverted band and its originating statistic event; rank calibration; seeded reproducibility; and reuse of precomputed bands. Finally, an empirical log-log timing slope guards against an accidental return to quadratic geometry. The test code is distributed with the notebook but is not reproduced here, so the printed appendix remains focused on the algorithm a reader is meant to study.

References

- [1] D. R. Barr and T. Davidson. A Kolmogorov–Smirnov test for censored samples. *Technometrics*, 15(4):739–757, 1973. doi:10.1080/00401706.1973.10489108.
- [2] R. H. Berk and D. H. Jones. Goodness-of-fit test statistics that dominate the Kolmogorov statistics. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 47(1):47–59, 1979. doi:10.1007/BF00533250.
- [3] S. Chattopadhyay, S. Sarkar, and A. K. Kuchibhotla. Inference for quantile-parametrized families via CDF confidence bands. In *Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence*, Proceedings of Machine Learning Research, vol. 337, pp. 1034–1053, 2026.
- [4] A. Dvoretzky, J. Kiefer, and J. Wolfowitz. Asymptotic minimax character of the sample distribution function and of the classical multinomial estimator. *Annals of Mathematical Statistics*, 27(3):642–669, 1956.
- [5] L. Dümbgen and J. A. Wellner. A new approach to tests and confidence bands for distribution functions. *Annals of Statistics*, 51(1):260–289, 2023. doi:10.1214/22-AOS2249.

- [6] R. G. Easterling. Goodness of fit and parameter estimation. *Technometrics*, 18(1):1–9, 1976. doi:10.1080/00401706.1976.10489394.
- [7] J. C. Frey. Optimal distribution-free confidence bands for a distribution function. *Journal of Statistical Planning and Inference*, 138(10):3086–3098, 2008. doi:10.1016/j.jspi.2007.12.001.
- [8] J. C. Frey, O. Marrero, and D. E. Norton. Minimum-area confidence sets for a normal distribution. *Journal of Statistical Planning and Inference*, 139(3):1023–1032, 2009. doi:10.1016/j.jspi.2008.06.005.
- [9] R. C. Littell and P. V. Rao. Confidence regions for location and scale parameters based on the Kolmogorov–Smirnov goodness of fit statistic. *Technometrics*, 20(1):23–27, 1978.
- [10] P. Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *Annals of Probability*, 18(3):1269–1283, 1990. doi:10.1214/aop/1176990746.
- [11] A. B. Owen. Nonparametric likelihood confidence bands for a distribution function. *Journal of the American Statistical Association*, 90(430):516–521, 1995. doi:10.1080/01621459.1995.10476543.
- [12] A. A. Salvia. Some fundamental properties of Kolmogorov–Smirnov consonance sets. *Technometrics*, 22(1):109–111, 1980. doi:10.1080/00401706.1980.10486107.